

Reliable Structured-Decision LLMs for Applicant Ranking with a Non-Autoregressive System One Model

Jessie Angelica¹, Hasanul Fahmi Zuhri^{1,2,*}, Rusdianto Roestam¹

¹Faculty of Computer Science, President University, Bekasi, Indonesia

²Faculty of Artificial Intelligence & Frontier Technologies, UNITAR International University, Petaling Jaya, Malaysia

Abstract—Large language model (LLM) pipelines for applicant ranking typically combine free-text assessment with iterative listwise comparison, which needs extra machinery against hallucinated claims and gives no built-in confidence signal. In September 2026, TypeSafe AI released Jev, the first “System One Model”: a non-autoregressive architecture that answers typed questions in one parallel pass and reports a calibrated confidence per answer, removing free-text generation and its hallucination risk by construction. No independent evaluation of this model class exists yet. The objective of this study is to close that gap by proposing and validating three architecture-level adaptations, namely criteria-grounded question design, structured score decomposition as a diagnostic signal, and multi-call ensemble averaging, that address the reliability problems Jev introduces when used without modification. The research method rebuilds an existing LLM-and-tournament applicant-ranking pipeline around Jev, evaluated on 346 shortlisted (job, applicant) pairs across 10 job profiles. The results show that criteria-grounded design raises per-requirement answer confidence from 0.694 to 0.890 ($n = 869$, $p < 0.001$, rank-biserial $r = 0.743$) and pooled seniority/education confidence from 0.650 to 0.705 ($p < 0.001$). A job profile’s score signal-to-noise ratio correlates with its ranking convergence ($\rho = 0.770$, $p = 0.0092$), indicating that per-requirement decomposition diagnoses genuine applicant proximity rather than model unreliability. Three-call ensemble averaging recovers ranking convergence from 0.901 to 0.940, close to the prior tournament-based system’s 0.957, using half as many model calls that run independently, in parallel. Internal coherence holds throughout ($\rho = 0.682$, $p < 0.001$). A non-autoregressive structured-decision model can therefore replace free-text assessment without losing reliability, provided its question design and aggregation match its architecture.

Keywords—Applicant Ranking, Confidence Calibration, Human Resources, Large Language Models, Structured Decision Models, System One Models.

I. INTRODUCTION

RECRUITMENT platforms increasingly use large language models (LLMs) to read resumes, summarize applicant fit, and compare applicants against a job profile, in place of manual screening or keyword search [1]–[4]. These systems are useful for the same reason they are risky: an LLM generates free text, and free text can assert a strength or weakness that the source resume never actually supports. Prior work responds to this hallucination risk with prompt-level constraints, retry-on-contradiction loops, and post-hoc auditing

[2]. It also combines free-text assessment with iterative listwise tournaments and Plackett-Luce aggregation, turning per-applicant judgments into a defensible ranking [1], [2]. Both kinds of mitigation exist because the underlying generation mechanism – next-token sampling over an open vocabulary – gives no structural guarantee against an ungrounded claim, and no native signal of how confident the model really is.

This paper asks whether a different generation paradigm – one that answers predefined, typed questions instead of generating open text – can replace the free-text assessment stage of an applicant-ranking pipeline without reintroducing the reliability problems it is meant to solve. Three gaps motivate this study. First, System One Models are new enough – Jev’s public release [8] predates this manuscript by days – that no independent, peer-reviewed evaluation exists yet, so every reliability claim currently comes from the vendor itself. Second, the recent literature on LLM calibration and self-consistency [5]–[7] targets autoregressive judges that produce a scalar score or a free-text verdict; none of it addresses a model whose primitive outputs are a bounded probability (Noul), a categorical distribution (Choice), or an ordinal distribution (Score) answered in one parallel pass, so its diagnostics and mitigations do not transfer directly. Third, the AI-hiring-ethics literature calls for transparency and accountability in automated hiring [12], [13] without asking whether a judgment model’s own output shape – free text versus typed, calibrated answers – affects either property.

This paper makes three architecture-level contributions, each validated empirically:

- 1) **Criteria-Grounded Question Design**: a methodology for writing Score-type questions with concrete, situational evidence descriptions instead of bare ordinal labels. A paired ablation shows this raises answer confidence for the per-requirement and pooled seniority/education Score questions, which have enough paired observations to detect the effect; the single, low- n overall-fit question does not reach significance on its own. A separate internal-coherence check confirms Score-derived judgments (per-requirement scores against overall fit) agree with each other, on the same assessed (job, applicant) pairs.
- 2) **Structured Score Decomposition as a Diagnostic Signal**: using the model’s per-requirement score granularity – each requirement scored on the same discrete-level-to-

*Corresponding author: fahmi.zuhri@unitar.my.

percent scale as overall fit, and aggregated the same way, by arithmetic mean across the ensemble calls – which a monolithic free-text judgment cannot offer, to detect and localize a pre-existing upstream shortlisting defect that a prose-only system could only expose through manual inspection.

- 3) **Multi-Call Ensemble Averaging:** an aggregation scheme that issues several independent Jev calls per applicant and combines them – arithmetic mean for score-valued answers, confidence-weighted majority vote for categorical answers. This recovers most of the ranking-order stability of an iterative tournament at roughly half the total model calls, without needing an iterative ranking process.

Section II reviews related work. Section III details the proposed pipeline’s architecture and the procedure behind each contribution. Section IV describes the experimental setup. Section V reports results, discusses them, and compares this system against the closest prior published system. Section VI concludes.

II. LITERATURE REVIEW

The system this paper extends combines LLM-driven free-text assessment with an active listwise tournament aggregated via Bayesian Plackett-Luce under Monte Carlo knowledge-gradient subset selection [1], [2]. Other systems apply LLMs to resume extraction and pointwise scoring inside retrieval-augmented or multi-agent architectures, without a comparative ranking step [3], [4], a trend recent work continues: Jahan et al. combine zero-shot classification with hybrid summarization for resume filtering [9], and Soni et al. use retrieval-augmented generation to score job compatibility [10]. None of this work, old or new, departs from free text or free-form JSON as its output mode, so all of it inherits the hallucination-mitigation burden [2] addresses with a retry loop, and none reports a native confidence that could distinguish a well-supported judgment from a guess.

A separate literature asks whether an LLM’s stated confidence reflects its true correctness probability. Yaldiz et al. [5] find RL from verifiable rewards improves accuracy but makes decision-making LLMs overconfident. Wang et al. [6] find structured-output reliability varies sharply across models and prompts under a fixed schema. Halder and Hockenmaier [7] document substantial self-inconsistency in LLM-as-a-judge frameworks, mitigated by averaging judgments; a recent survey by Gu et al. catalogs reliability strategies for such judges, but every one surveyed still targets a free-text or scalar-score judge [11]. All of this, including the survey, treats the model as autoregressive, estimating confidence from token probabilities or resampling – an assumption Jev’s natively-trained, non-autoregressive confidence field does not share, so these findings cannot be assumed to transfer, which this paper verifies independently.

A third body of work examines AI hiring at the organizational and ethical level, not the architecture level: Soleimani et al. find, from interviews with 39 HR professionals and AI developers, that reducing bias in AI recruitment requires

understanding the mechanism behind a judgment, not just auditing outputs [12], and Kwok et al. apply the OECD AI Principles to GenAI-assisted hiring, flagging transparency and accountability as the least-satisfied [13] – neither asks whether a judgment model’s output shape itself affects either property. No paper reviewed here evaluates a System One Model; to the authors’ knowledge this is the first independent study of this model class, whose own claims live only in vendor-controlled or community sources at the time of writing – the announcement [8], an integration guide [14], a community write-up [15], and a release summary [16] – none peer-reviewed. This work sits in applicant ranking, already served by a directly comparable free-text predecessor [1], [2], making an architecture-level comparison possible, not just a benchmark-level one.

A. Background: Jev as a System One Model

Jev is explicitly not a conventional language model. TypeSafe AI’s own integration documentation states plainly: “Jev is not a language model. You give it a text and typed questions, and it answers each one with a confidence. It does not write text” [17]. Where an LLM samples output tokens autoregressively, Jev takes a text state and a fixed set of typed questions (Noul, Choice, or Score) and returns, in one parallel pass, a calibrated probability distribution over each question’s possible answers; no text is generated, and TypeSafe prices output tokens at zero because none are produced [8]. TypeSafe places Jev in a new class it calls a “System One Model,” trained with a proprietary method the vendor names Reinforcement Learning for Calibrated Decisions (RLCD) and served through a purpose-built parallel sampler [8]. The underlying neural architecture – whether it is encoder-only, decoder-only, or some other design – is not disclosed in any vendor or third-party source consulted for this paper. Jev’s observed behavior is nonetheless consistent with a classifier-style, encoder-like model: it maps a fixed input directly to a distribution over a closed set of predefined answers in a single forward pass, rather than generating a variable-length sequence token by token. This is an inference from behavior, not a confirmed architectural fact, since TypeSafe has not published the underlying design. Jev was created by Diogo Almeida, TypeSafe AI’s co-founder and CEO, who “co-invented RLHF and InstructGPT, the methods that lead to ChatGPT and GPT4” [18] before founding TypeSafe, which launched Jev publicly on September 15, 2026 with a \$40M seed round [8]. As used in this paper (`jev-latest`), Jev accepts up to 64,000 tokens per request, 32,000 of which may be the state plus the single longest question [19], responds in 70–500ms end-to-end, and is priced at \$0.042 per million input tokens with output tokens free [8].

III. METHODOLOGY

A. Pipeline Overview

Fig. 1 shows the end-to-end pipeline, with C1–C3 marking where each of the three contributions above intervenes. Its first three stages are unchanged from the prior system [2], because none of them generate free text, so none carry the reliability

risk this paper addresses. Job profiles are loaded as plain text and resumes are parsed from PDF; an LLM (Qwen2.5-14B-Instruct) extracts a normalized skill list for each job profile and applicant, including any stated certification, seniority, and education requirement; and an applicant is shortlisted only if enough of a job profile’s technical skills match theirs, by cosine similarity, above a fixed threshold.

The pipeline changes at the next stage, because that is exactly where the prior system’s free text lived: instead of a free-text assessment ranked by an iterative tournament, it sends n independent calls to Jev per shortlisted pair. Each call gives Jev one state (job title, description, extracted skills, resume text) and a fixed set of typed questions answered in one pass: an overall-fit Score, a hiring-recommendation Choice, a minimum-qualifications Noul, one Score per technical requirement, one Score per stated seniority or education requirement, and one Noul per stated certification. Every Score question – overall-fit, per-requirement, or per-qualification – offers a fixed set of ordinal levels (Section III-B); Jev selects one level per call, which is then linearly mapped to a 0–100 value for reporting – the 0–100 range is this mapping’s resolution, not a continuous per-call judgment. The n calls are combined (Section III-D), applicants are ranked by the combined score, and a diagnostic check (Section III-C) reads the ranked pool’s per-requirement scores afterward for upstream errors. To be precise: skill/JD extraction runs on a conventional autoregressive LLM, but Jev does not – it samples no tokens and writes no free text, only typed answers in one forward pass.

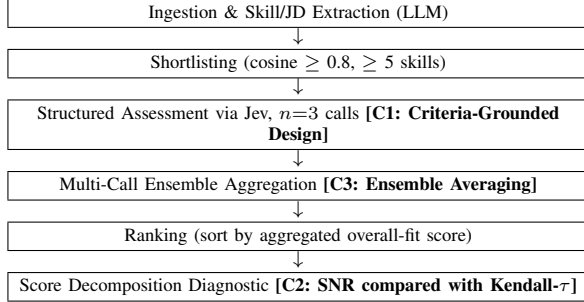


Fig. 1. End-to-end pipeline architecture, annotated with the three contributions (C1–C3) at their point of intervention.

B. Criteria-Grounded Question Design

An initial implementation defined every Score question’s five levels with bare numeric labels (“0,” “25,” “50,” “75,” “100”), with no description of what each level meant. This is why it performed poorly: asked to pick a level with no stated meaning, Jev had to guess where the boundary between levels fell, and guessing lowers reported confidence. The fix was to describe each level as a concrete, situational fact instead of a bare number – for example, distinguishing a requirement “mentioned only as a bare skill-list item” from one “used substantively in a real role or project with clear responsibility,” so the model is matching evidence to a description rather than guessing at a boundary.

To check that this fix, not ordinary sampling noise, caused the confidence gain, a paired ablation compared the two

designs on the identical input: for a sampled subset of (job, applicant) pairs already assessed under the concrete-criteria design, a second call was issued against the exact same state but with the original bare-label design, and confidence was compared pairwise, question by question. The concrete side of each pair is the confidence already recorded from that pair’s production assessment – a mean across 3 independent calls – while the bare-label side is a single fresh call; this asymmetry shrinks the concrete side’s variance relative to the bare-label side’s but does not bias either side’s mean, so it cannot manufacture the gain reported here. The primary comparison is the many per-technical-requirement questions, the group with enough paired observations per job profile to detect the effect reliably; a Wilcoxon signed-rank test with a rank-biserial effect size was applied.

The same fix – naming evidence instead of leaving the model to guess – generalizes to the seniority and education Score questions, which had the same problem in a different shape: their five levels were initially given the same bare ordinal labels (“0,” “25,” “50,” “75,” “100”) as the unfixed Score questions above, rather than the concrete evidence descriptions used for technical requirements. Rewriting the levels to name concrete evidence – explicit years or seniority level, domain match, or role match for seniority; a matching field, a matching degree level, or both, for education – follows the same logic as the technical-requirement fix, tested with a second paired ablation on all (job, applicant) pairs whose job profile states either requirement. One alternative was rejected: generating levels per job profile with an LLM, since that reintroduces the free-text generation this pipeline exists to remove, and would make scores for the same requirement incomparable across differently worded job profiles.

C. Structured Score Decomposition as a Diagnostic Signal

Because Jev reports a separate score per extracted requirement rather than a single free-text verdict, a job profile’s shortlisted pool can be inspected at the level of individual skills, not only an aggregate score. This granularity is used diagnostically in two ways.

The first way asks whether an unstable ranking means the model is unreliable, or simply that its applicants score too close together to rank apart – the two cases look different in the data: genuinely differing applicants should spread out more than the noise of asking the same one the same question twice, while similar applicants should not. This is a signal-to-noise ratio, computed per job profile as

$$\text{SNR} = \frac{\text{std}_i(\bar{s}_i)}{\frac{1}{N} \sum_{i=1}^N \text{std}_k(s_{i,k})}, \quad (1)$$

where $s_{i,k}$ is applicant i ’s overall-fit score on call k (of K), $\bar{s}_i = \frac{1}{K} \sum_{k=1}^K s_{i,k}$ is that applicant’s mean score, and N is the shortlisted-pool size; the numerator is between-applicant signal, the denominator is mean within-applicant noise. This ratio is correlated (Spearman) against that profile’s Kendall- τ (Section III-D): if low convergence tracks low SNR, that supports genuine score proximity over model unreliability.

The second diagnostic use asks a different question: did the shortlisting stage, before Jev is even involved, let through

applicants who do not actually fit the role? A requirement is informative for this check only if the shortlisted pool’s own median score on it is reasonably high – meaning most peers in that pool do have credible evidence for it. An applicant scoring near-zero on most informative requirements, despite having been shortlisted, is flagged: not because Jev judged them harshly, but because their peers largely could clear a bar this applicant could not.

D. Multi-Call Ensemble Averaging

A single Jev call is a single sample, and any sample carries noise that can shift a ranking. Because calls are independent and cheap, the fix is to average that noise away: the pipeline issues n calls per pair – three by default, matching [2]’s stability-repeat count – combined per field by whatever matches its type: numeric fields by arithmetic mean, the categorical recommendation by majority vote (ties toward higher mean confidence), and binary answers by simple majority, defaulting to the conservative (negative) outcome on a tie.

Averaging could also just hide real disagreement rather than remove noise, so this needs a check: ranking convergence was measured both from single, unaggregated calls (Kendall- τ across three per-pair rankings per job profile) and from three independent three-call averages (Kendall- τ across three trial rankings, each trial’s own three-call average). If averaging only hid disagreement, the second number would not consistently beat the first.

IV. EXPERIMENTAL SETUP

The pipeline is evaluated on the same corpus and shortlist configuration as the prior system [2]: 501 resumes across 10 synthetically authored job profiles, shortlisted under a cosine-similarity threshold of 0.8 and a minimum of five matched distinct technical skills per job profile, yielding 346 shortlisted (job, applicant) pairs in the run reported here. As shown in Table I, these settings mirror [2] exactly and were held fixed across every evaluation in this paper, isolating the assessment-stage change as the one independent variable between the two systems. Four evaluation procedures are applied: test-retest reliability (three independent single calls per pair, measuring score variance and recommendation agreement); internal coherence (per-requirement scores against overall fit score via Spearman correlation); ranking convergence (the Kendall- τ procedure of Section III-D, from single calls and from three-call averages); and the criteria-design ablations of Section III-B. All statistical tests are two-sided except where a directional hypothesis is stated explicitly. Every reported effect size accompanies, rather than substitutes for, its significance test.

V. RESULTS AND DISCUSSION

A. Test-Retest Reliability and Internal Coherence

Across $346 \text{ pairs} \times 3 \text{ repeats}$, the overall-fit score’s within-pair standard deviation (0–100 scale) has mean 0.82 (median 0.66), and the recommendation is identical across all 3 repeats for 95.1% of pairs – both indicate low single-call noise before

TABLE I
PIPELINE CONFIGURATION.

Parameter	Value
Assessment model	Jev (jev-latest), TypeSafe AI
Skill/JD extraction model	Qwen2.5-14B-Instruct (4-bit, Ollama)
Skill embedding model	sentence-transformers/all-MiniLM-L6-v2
Corpus	501 resumes, 10 job profiles
Shortlisted (job, applicant) pairs	346 (338–346 run-to-run)
Skill-match threshold / min. matches	cosine ≥ 0.8 / ≥ 5 job skills
Independent calls averaged per assessment (n)	3
Score question levels	5 (concrete situational criteria)
Test-retest repeats per pair	3
Ranking-convergence trials (averaged-score check)	3×3 calls
Criteria-design ablation sample	40 pairs (Score); 169 pairs (seniority/education Score)

any aggregation. Per-requirement scores correlate with the overall fit score ($\rho = 0.682$, $p < 0.001$, $n = 346$).

Why is $\rho = 0.682$ significant yet short of 1.0? Because of how the numbers are produced, not a reliability defect: Jev answers the overall-fit and every per-requirement question as independent items in the same call – overall fit is never computed from the per-requirement scores, so they are two separate readings of the same evidence. Overall fit also weighs things an unweighted skill average cannot, such as seniority fit, and single-call noise pulls the correlation down further. A correlation near zero, not below 1.0, would signal incoherent judgments; $\rho = 0.682$ is what two related but distinct judgments should look like.

B. Criteria-Grounded Question Design

TABLE II
CRITERIA-DESIGN ABLATIONS: CONCRETE COMPARED WITH BARE-LABEL CRITERIA, PAIRED BY (JOB, APPLICANT) PAIR.

Question group	n	Mean conf. (concrete / other)	p (r)
Per-requirement (technical)	869	0.890 / 0.694	< 0.001 (0.74)
Seniority	66	0.583 / 0.585	0.597 (0.17)
Education	146	0.760 / 0.679	< 0.001 (0.48)
Pooled (seniority + education)	212	0.705 / 0.650	< 0.001 (0.38)

Each row pairs the same question asked twice under the two criteria designs and reports mean confidence under each plus the Wilcoxon result. As shown in Table II, the many per-requirement questions show a large, highly significant confidence gain ($r = 0.743$, $p < 0.001$, $n = 869$). The same principle holds for the seniority and education Score questions, reaching significance pooled and for education alone but not seniority alone ($p = 0.597$, $n = 66$, concrete mean 0.583 against bare-label 0.585); the direction matches the per-requirement result, but the pooled effect is smaller ($r = 0.38$ compared with $r = 0.743$) – unlike the per-requirement questions, which pool many distinct skill-match judgments per pair, seniority and education each contribute a single judgment per pair, so a non-significant seniority effect on its own pulls the pooled effect size down. The seniority null result is not simply noise: education’s criteria check explicit, unambiguous facts a resume either states or does not (a degree, a field), which concrete wording sharpens directly,

while seniority’s criteria require inferring years of experience and domain relevance at once – a harder judgment that bare ordinal labels do not meaningfully worsen, so criteria wording alone has less to fix there.

C. Structured Score Decomposition Diagnostic

TABLE III
SIGNAL-TO-NOISE RATIO (SNR) AND RANKING CONVERGENCE
(KENDALL- τ) BY JOB PROFILE, SINGLE-CALL AND 3-CALL-AVERAGED.

Job profile	n	SNR	τ_1	τ_3
it-security-engineer	16	5.63	0.778	0.900
data-scientist-eng.	27	5.72	0.848	0.890
backend-engineer	22	10.05	0.896	0.954
frontend-engineer	13	13.98	0.880	0.897
devops-engineer	58	12.51	0.923	0.956
full-stack-engineer	33	10.63	0.914	0.944
data-engineer	76	17.79	0.937	0.958
ui-ux-designer	23	24.49	0.947	0.984
java-developer	21	13.33	0.949	0.949
cloud-engineer	57	17.30	0.933	0.963
Mean (all 10)	–	–	0.901	0.940

Table III gives each profile’s SNR (Equation 1) alongside its Kendall- τ . SNR and τ_1 correlate positively and significantly across the 10 profiles ($\rho = 0.770$, $p = 0.0092$) – support for reading low convergence as close-scoring applicants, not an unreliable judge. The diagnostic is also demonstrated concretely: the informative-requirement check flagged 8 of 76 data-engineer applicants (11%) near-zero specifically on Python, and 2 of 33 full-stack-engineer applicants (6%) near-zero on REST APIs and Node.js despite adequate scores elsewhere – evidence the skill-matching stage, not Jev, admitted these applicants. With $n = 10$ profiles, the correlation is suggestive, not conclusive, until replicated on more job profiles.

D. Multi-Call Ensemble Averaging

τ_1 is from single unaggregated calls, τ_3 from three-call averages (Section III-D). As shown in Table III’s $\tau_1 \rightarrow \tau_3$ columns, three-call averaging raises mean convergence from 0.901 to 0.940, with 9 of 10 profiles improving (java-developer is unchanged at 0.949; largest gain: it-security-engineer, 0.778 $\rightarrow$ 0.900) – improvement across nearly every profile confirms the aggregation recovers genuine stability, not just smoothing over disagreement.

E. Efficiency

Across all 1,038 calls in the test-retest data (346 pairs $\times$ 3 repeats), a single Jev call takes a mean of 1.20s (median 1.18s, p95 1.64s, range 0.73–3.33s), for a total of 20.7 minutes of compute time across the full corpus – call latency only, not including skill/JD extraction. Table IV breaks this down per job profile; mean latency ranges from 1.03s (ui-ux-designer) to 1.33s (cloud-engineer), with no clear relationship to shortlisted-pool size: the largest pool (data-engineer, 228 calls) is among the slower profiles, but the second-largest (devops-engineer, 174 calls) sits near the middle.

TABLE IV
MEAN JEV CALL LATENCY BY JOB PROFILE ($n = 1,038$ CALLS TOTAL).

Job profile	Calls	Mean latency (s)
ui-ux-designer	69	1.03
full-stack-engineer	99	1.09
frontend-engineer	39	1.11
it-security-engineer	48	1.13
devops-engineer	174	1.13
data-scientist-engineer	81	1.15
backend-engineer	66	1.16
data-engineer	228	1.28
java-developer	63	1.31
cloud-engineer	171	1.33

TABLE V
COMPARISON AGAINST THE CLOSEST PRIOR PUBLISHED SYSTEM [2].

Dimension	Prior system [2]	This work
Assessment	Free-text strengths/weaknesses	Typed Score/Choice/No/Unanswered in one parallel pass
Output shape	+ retry-on-contradiction Free prose, validated against a schema after generation	Predefined typed answers; schema violation impossible by construction
Hallucination handling	Live contradiction-check-and-regenerate loop	Not applicable: no free text to hallucinate
Identifier robustness	Positional tokens + schema-constrained ranking + retry	Not needed: one call per applicant, no identifier passing
Ranking	Iterative MC-KG tournament + Plackett-Luce	Direct sort on aggregated score
Convergence	0.957	0.940 (3-call average)
Calls per (job, applicant) pair	1 assessment + share of the job’s tournament rounds	1 (all questions answered in parallel)
Total calls (corpus)	$\approx 2,061$ (348 assess. + 1,713 tournament)	1,038 (346 $\times$ 3), 0 tournament
Generation model	Qwen2.5-14B-Instruct, 4-bit, local via Ollama	jev-latest, hosted by Type-Safe AI
Confidence signal	Faithfulness 0.880 (entailment)	Calibrated confidence 0.890 (requirement questions); different construct
Explanation	Strengths/weaknesses narrative	Score + confidence, no narrative

F. Comparison with the Prior Tournament-Based System

As shown in Table V, this system is not better in every way. Its ranking is slightly less stable, and it does not improve groundedness either: Faithfulness checks whether a written claim is supported by the resume, while per-answer confidence measures how stable a decision is, so the two numbers measure different things and neither one proves the system is “more accurate.” Where it does win is cost: it uses half the calls, needs no tournament infrastructure, and its output cannot break the schema by construction, whereas the prior system could only reduce such failures in practice. It also offers something the prior system had no way to provide: a native, calibrated confidence for every answer. One further advantage has no number attached: the prior system’s 1,713 tournament calls form a sequential chain in which each iteration waits for the previous one to finish, whereas this system’s calls are independent and run in parallel.

VI. CONCLUSION

This paper replaced the free-text assessment and iterative tournament-ranking stages of an existing pipeline with Jev, TypeSafe AI’s first System One Model, and validated three adaptations – criteria-grounded question design, structured score decomposition as a diagnostic signal, and multi-call ensemble averaging – each motivated by a reliability problem observed in practice. As Table V makes clear, the result is not a uniform improvement over the prior system: it trades

a little ranking stability and all free-text interpretability for roughly half the model calls, no tournament infrastructure, and a native calibrated confidence the prior system never had. Criteria-grounded design is strongly validated for the per-requirement Score questions ($r = 0.743$, $p < 0.001$, $n = 869$) and, with a smaller pooled effect, for the seniority and education Score questions ($r = 0.38$); the single, low- n overall-fit question does not reach significance on its own, though it is answered by the same fixed criteria as the per-requirement questions it is paired with in every call. Score decomposition’s diagnostic value is confirmed quantitatively, not just anecdotally, surfacing a shortlisting-stage defect the prior system could only have found by manual reading.

A limitation is that the SNR-convergence correlation in Section V-C has only $n = 10$ points: SNR and convergence are properties of a whole job profile’s pool, not an individual applicant, so this correlation has at most one point per job profile regardless of applicant count – significant, but suggestive until replicated on more job profiles. A second is that this work trades away the prior system’s free-text interpretability: output is a score and a confidence value, not a narrative explaining why, and per-requirement scores only partly substitute – a direct trade-off against the reliability gains reported here. A third is that the shortlisting stage, inherited unchanged from the prior system, treats every extracted skill equally: it counts matches toward a minimum number without distinguishing a role’s core stack from peripheral, generic skills, so an applicant can be admitted by matching only utility skills (e.g., Docker, Kubernetes, AWS) while scoring near-zero on the role’s core requirements – the full-stack-engineer case Section V-C surfaces. Score decomposition detects this defect but does not correct it, since doing so would require modifying the upstream stage both pipelines share.

The future work is a replication of the SNR-convergence correlation on more job profiles, since its current support rests on only $n = 10$ points. A second is a lightweight explanation layer – surfacing the lowest-scoring items as a structured rationale – to partially restore the interpretability this work trades away, without reintroducing free text. A third is separating a role’s core technical skills from peripheral, generic ones during JD skill extraction, so the shared shortlisting stage stops admitting applicants who match only utility skills.

ACKNOWLEDGMENT

The authors acknowledge the use of NVIDIA GeForce RTX 4070 Ti SUPER GPU hardware for the local skill/JD-extraction stage, and the open-source LangGraph, sentence-transformers, and PyTorch libraries.

REFERENCES

- [1] K. A. Yuksel, A. B. Anees, A. Elneima, S. Hewavitharana, M. Al-Badrashiny, and H. Sawaf, “Agentic AI for Human Resources: LLM-Driven Candidate Assessment,” in *Proc. 19th Conf. Eur. Chapter Assoc. Comput. Linguistics (EACL)*, vol. 3: System Demonstrations, Rabat, Morocco, Mar. 2026, pp. 341–348.
- [2] J. Angelica and H. F. Zuhri, “Hallucination-Aware Self-Correction for LLM-Based Applicant Assessment,” unpublished manuscript, 2026.
- [3] A. C. Tran and N. C. Tran, “A recruitment support system based on Large Language Models and Retrieval-Augmented Generation,” *Procedia Computer Science*, vol. 269, pp. 259–268, 2025.
- [4] F. P.-W. Lo, J. Qiu, Z. Wang, H. Yu, Y. Chen, G. Zhang, and B. Lo, “AI Hiring with LLMs: A Context-Aware and Explainable Multi-Agent Framework for Resume Screening,” arXiv:2504.02870, 2025.
- [5] D. N. Yaldiz, E. Spiliopoulou, Z. Qi, S. Varia, S. Doss, and N. Pappas, “Balancing Classification and Calibration Performance in Decision-Making LLMs via Calibration Aware Reinforcement Learning,” arXiv:2601.13284, 2026.
- [6] G. Wang, J. Yu, X. Zhang, D. Jiang, Y. Song, T. Deb, X. Liu, and P. He, “STED and Consistency Scoring: A Framework for Evaluating LLM Structured Output Reliability,” arXiv:2512.23712, 2025.
- [7] R. Haldar and J. Hockenmaier, “Rating Roulette: Self-Inconsistency in LLM-As-A-Judge Frameworks,” in *Findings of the Assoc. Comput. Linguistics: EMNLP 2025*, 2025.
- [8] D. Almeida, “Introducing System One Models & Jev,” Type-Safe AI Company Blog, Sep. 15, 2026. [Online]. Available: <https://typesafe.ai/blog/introducing-system-one-models-and-jev>
- [9] N. Jahan, M. B. Uddin, M. S. Arefin, C. Mohammad Mehedi, F. Tasnim, and K. E. Delowar, “Enhancing Resume Screening Through Multi-stage LLM Classification and Hybrid Summarization,” in *Proc. 3rd Int. Conf. Big Data, IoT and Machine Learning (BIM 2025)*, Lecture Notes in Networks and Systems, vol. 1798, Springer, Cham, 2026.
- [10] K. S. Soni, P. Talati, J. Mandani, D. Garg, and P. Goel, “Intelligent Resume Evaluation Using LLMs: Skill Gap Identification and Job Compatibility Scoring,” in *Information Systems for Intelligent Systems (ISBM 2025)*, Smart Innovation, Systems and Technologies, vol. 468, Springer, Cham, 2026.
- [11] J. Gu et al., “A survey on LLM-as-a-judge,” *The Innovation*, vol. 7, no. 6, art. 101253, 2026.
- [12] M. Soleimani, A. Intezari, J. Arrowsmith, D. J. Pauleen, and N. Taskin, “Reducing AI bias in recruitment and selection: an integrative grounded approach,” *Int. J. Human Resource Management*, vol. 36, no. 14, 2025.
- [13] L. Kwok, N.-K. Chuang, and Y.-K. Huang, “Staying Ahead: A Critical Assessment of the GenAI-Assisted Hiring Process Using the OECD AI Principles,” *J. Hospitality & Tourism Research*, 2025.
- [14] S. Runkle and H. Lovell, “Building a Harness with Jev,” LangChain Blog, Sep. 17, 2026. [Online]. Available: <https://www.langchain.com/blog/building-a-harness-with-jev>
- [15] P. Burnhill, “TypeSafe Jev – Project Knowledge Resource,” GitHub Gist, Sep. 16, 2026. [Online]. Available: <https://gist.github.com/pjburnhill/adf8d28efcad9df037bfdece178ef965>
- [16] “Jev – TypeSafe’s model returns typed decisions, not text,” AI/TLDR, Sep. 15, 2026. [Online]. Available: <https://ai-tldr.dev/releases/typesafe-jev/>
- [17] “TypeSafe (Jev),” Pydantic AI Documentation. [Online]. Available: <https://pydantic.dev/docs/ai/models/typesafe/>
- [18] “Team,” TypeSafe AI, accessed Sep. 2026. [Online]. Available: <https://typesafe.ai/team>
- [19] “Models,” TypeSafe AI Documentation, accessed Sep. 2026. [Online]. Available: <https://docs.typesafe.ai/models>